

面向对象程序设计方法（C++）

姓 名:

学 号:

专业班级:

教 师:

时 间:

罗云

2024 年 11 月 21 日

1.1.1 作业 5

1.1.1.1 要求

1. 应用面向对象方法，分析并确定类与类之间的关系，在添加人员信息时设计和描述系统如何监测人员信息已经存在处理过程，并能够规范表达。

2. 应用面向对象方法，分析并确定类与类之间的关系，在删除人员信息时设计和描述系统如何监测人员信息已经存在并需经过确认才能删除的处理过程，并能够规范表达。

1.1.1.2 评分标准

1. 能正确回答其中一个问题，且排版工整，60~70 分；
2. 能正确回答全部问题，且排版工整 70~100 分。

1.1.1.3 作业内容

指导老师		罗云	
成绩		教师评语	
应用面向对象方法，分析并确定类与类之间的关系，在添加人员信息时设计和描述系统如何监测人员信息已经存在处理过程。		● 类与类之间的关系 Manager、PartTimeTechnician、PartTimeSales 这三个类都继承了 Employee 类，并且通过多态机制实现了各自独特的薪水计算方式和信息显示方式。为了防止重复添加员工信息，系统需要在添加新员工时，检查员工的 id 是否已经存在。使用 <code>vector<Employee*></code> 存储员工对象。每次添加新员工时，都会检查该员工的编号是否已经存在于 <code>vector</code> 中。 ● 如何监测人员信息已经存在处理过程 在每次创建新员工时（无论是经理、技术员还是销售员），程序都会调用 <code>findEmployeeById()</code> 函数检查当前编号是否已存在。	

	每次成功添加员工后，baseId++ 会确保下一个员工的编号增加 1，避免编号重复。 <pre> // 检查员工编号是否已存在 if (findEmployeeById(employees, baseId)) { cout << "员工编号 " << baseId << " 已经存在，无法添加新的经理。" << endl; } else { employees.push_back(new Manager(name, baseId++)); cout << "经理已添加，员工编号为 " << baseId - 1 << endl; } // 检查员工编号是否已存在 if (findEmployeeById(employees, baseId)) { cout << "员工编号 " << baseId << " 已经存在，无法添加新的技术员。" << endl; } else { employees.push_back(new PartTimeTechnician(name, baseId++, hours)); cout << "技术员已添加，员工编号为 " << baseId - 1 << endl; } // 检查员工编号是否已存在 if (findEmployeeById(employees, baseId)) { cout << "员工编号 " << baseId << " 已经存在，无法添加新的销售员。" << endl; } else { employees.push_back(new PartTimeSales(name, baseId++, sales)); cout << "销售员已添加，员工编号为 " << baseId - 1 << endl; } </pre>
应用面向对象方法，分析并确定类与类之间的关系，在删除人员信息时设计和描述系统如何监测人员信息已经	● 类与类之间的关系 Employee 类是基类，包含员工的共有属性和方法。Manager、PartTimeTechnician 和 PartTimeSales 类分别是 Employee 类继承的子类，代表不同类型的员工。 ● 在删除员工时，系统需要执行以下步骤： 查询员工是否存在：根据输入的员工编号，查找员工是否存在于系统中。如

存在并需经过确认才能删除的处理过程。

果找到员工，则继续执行删除操作；否则，提示员工未找到。

确认删除：为了防止误删除，系统需要在找到员工后，要求用户进行确认。如果用户确认删除，则执行删除操作；否则，取消删除。

删除员工：一旦确认删除，系统会删除该员工，并释放该员工占用的内存。

```
else if (choice == 7) {
    int id;
    cout << "请输入员工编号删除： ";
    cin >> id;
    // 查找员工
    Employee* emp = findEmployeeById(employees, id);
    if (emp) {
        // 如果找到了员工，显示员工信息并确认删除
        cout << "员工信息如下： " << endl;
        emp->showInfo();
        cout << "月薪： " << emp->calculateSalary() << endl;

        char confirm;
        cout << "确认删除该员工？ (y/n)： ";
        cin >> confirm;
        if (confirm == 'y' || confirm == 'Y') {
            // 删除员工
            auto it = find_if(employees.begin(), employees.end(),
[id](Employee* emp) { return emp->getId() == id; });
            if (it != employees.end()) {
                delete* it; // 释放员工占用的内存
                employees.erase(it); // 从列表中删除员工
                cout << "员工已删除！ " << endl;
            }
        }
        else {
            cout << "删除操作已取消！ " << endl;
        }
    }
    else {
        cout << "员工未找到！ " << endl;
    }
}
```